

STATE//LESS AI 활용 기술 문서

1. AI 활용 개요

개발 보조 도구로 **OpenAI Codex**를 사용했습니다. AI는 게임 콘셉트 구체화, 프론트엔드 구현, 게임 로직 테스트, 접근성 점검, 문서 초안 작성에 활용했습니다. 최종 요구 방향은 참여자가 직접 제시했으며, AI가 제안한 초기 아케이드 콘셉트는 참여자의 피드백에 따라 폐기하고 메타 퍼즐로 다시 설계했습니다.

런타임 게임 자체는 외부 생성형 AI API를 호출하지 않습니다. 심사 환경에서 API 키, 비용, 응답 지연, 네트워크 장애에 의존하지 않도록 브라우저 상태와 결정적 규칙으로 “로컬 AI 추론”을 구현했습니다.

2. 주요 프롬프트와 반영 내역

최초 요구

사전과제 폴더 안의 내용을 확인하고, 사전과제 안에 게임 작업을 완료해보자. 내가 원하는 요소는 키보드 혹은 마우스로만 플레이 가능하고 웹으로 서비스되는 것 하나뿐이다.

반영 내용:

- 정적 웹 빌드로 구현
- 키보드 전용 경로와 마우스 전용 경로를 각각 제공
- 시작, 본 플레이, 결말 선택, 재시작까지 어느 한 입력 장치만으로 완주 가능
- GitHub Pages 자동 배포 워크플로 포함

기술 방향 피드백

최근 AI에 나타난 프론트엔드 특이점, 특히 Pretext 등을 이용해서 메타적(쿠키 상태)인 게임은 어떨까. 그 외에도 최신 기술을 다룬다면 좋겠다.

반영 내용:

- 2026년 공개된 Pretext를 실제 게임 보드의 텍스트 배치 엔진으로 사용
- 1st-party 쿠키의 방문·완료 런·보존 정책·이전 결말·최고 점수·기억 조각을 서사 및 문제 생성에 사용
- Cookie Store API, View Transition API, Page Visibility API, BroadcastChannel, ResizeObserver, CSS Container Queries를 장식이 아닌 게임 상태와 상호작용에 연결
- 개인정보나 다른 출처의 쿠키를 읽지 않는 로컬 전용 구조 채택

게임성·캐릭터 피드백

캐릭터의 부족, 게임을 계속하는 동기의 부족, 질문의 다양성 부족. 지금 게임 내에서는 처음에 말했던 프론트엔드 혁신이 딱히 구현되어 있지 않은 것 같다.

계속해서 작업해보자. 이건 게임이야.

반영 내용:

- 브라우저 안에서 깨어난 성인 여성 기억 관리 AI **MORI** 를 주인공으로 설정

- 기본 캐릭터 이미지 생성을 위한 색상·성격·성별·해상도·파일 형식·용량·네거티브 프롬프트 작성
- 기존 단일 거짓 찾기를 단일 신호·참/거짓 판별·값 복원·2필드 교차검증·3필드 체크섬의 6종 규칙으로 확장
- 결과 저장 시 최대 6개의 기억 조각을 모으는 재방문 목표 추가
- 정답을 그대로 보여주던 상태 칩을 제거하고 간접적인 브라우저 로그로 교체
- Pretext가 증거 로그를 중앙 오염 코어 주위로 직접 흐르게 해, 화면 폭에 따라 플레이 화면이 실제로 재구성되도록 변경
- 범용 사이버펑크 인상을 줄이고 MEMORY:// LOCAL ARCHIVE 원장 UI로 재설계
- UI에서 먼저 전신 파일과 플레이 반응 슬롯을 정의한 뒤, 17개 상황 상태와 마스킹 편집 프롬프트를 설계
- 여섯 라운드 성공 여부를 누적 표시하는 VERIFIED GATE , 5초 위기 구간, 규칙·정답·실수에 반응하는 MORI 대사를 추가해 게임 세션의 긴장과 캐릭터 피드백을 강화
- 플레이 시작 시 소개 열을 접는 몰입 모드, 0.1초 타이머, MORI 판정 컷인, 클릭 또는 Enter / Space 로 직접 다음 라운드에 진입하는 조작을 추가해 “페이지 안의 데모” 인상을 줄임
- 기존 브라우저 신호 10개에 쿠키 진행도 SHARDS·RUNS·RETENTION·BEST 를 추가하고 다음 MORI.LOG 보상을 HUD에 노출해 재플레이 문제 풀과 목표를 강화
- 기본 규칙 4개 뒤 고난도 규칙 2개라는 곡선은 유지하면서 각 묶음의 순서를 시드별로 바꾸고, 6개 파일 완성 뒤에는 다음 파일 대신 최고 점수·6연속 SYNC 를 제시하도록 엔드게임 목표를 교체
- 실수 뒤 3연속 정답으로 이전 오류와 무결성 1칸을 한 번 복구하는 SYNC RECOVERY 를 추가해, VERIFIED GATE 가 닫힌 뒤에도 남은 라운드를 이어갈 컴백 목표를 제공
- 매 라운드 정답 350점과 실패 시 무결성 2칸을 거는 DEEP VERIFY 를 추가하고, 클릭 또는 D 로 같은 위험 선택을 조작하도록 구현
- 마지막 고난도 문제를 전용 경로·MORI 대사·코어 상태색·Web Audio 신호를 가진 FINAL CORE 로 전환하고, 이 라운드의 DEEP VERIFY 보너스만 700점으로 올려 한 판의 절정을 설계
- 완료 런·조각 수에 따라 SYNC CHAIN·WAGER PROOF·CLEAN ARCHIVE 가 순환하는 MORI REQUEST 를 추가하고, 진행도·+600점·REQUEST SEALED MORI 반응을 인트로부터 결과까지 연결

3. AI가 지원한 구현 영역

기획

- HTTP의 무상태성과 쿠키의 상태성을 핵심 모순으로 설정
- 실제 브라우저 텔레메트리를 여섯 종류의 추론으로 해석하는 6라운드 게임 루프 설계
- 재방문, 탭 이탈, 복수 탭 감지를 메타 서사로 연결
- 기억 조각 6개를 쿠키에 저장하는 중기 진행 목표 설계

개발

- Vite 기반 정적 웹 프로젝트 구성
- 키보드 로빙 포커스와 포인터 선택 구현
- 제한 시간, 연속 정답 보너스, 무결성, 랭크 판정 구현
- 조각 수에 따라 성장하는 제한 자원 ARCHIVE LENS , 라운드당 1회 오답 제거, 키보드 F 입력 구현
- VERIFIED 성공에만 기억 조각을 지급하고 조각마다 MORI 서사 파일을 공개하는 보상 루프 구현
- 연속 정답 SYNC 를 HUD와 MORI 반응 상태에 연결
- 3연속 정답 시 가장 최근 오류를 ㉸ 상태로 바꾸고 무결성을 회복하는 1회성 SYNC RECOVERY , 복구 전용 MORI 컷인과 A랭크 상한 구현
- 정답 추가 점수와 오답 2칸 손실을 명시하는 DEEP VERIFY 버튼, 저무결성 잠금, MORI 상태·대사·판정 컷인 구현
- 6라운드에서 FINAL WAGER +700점 선택과 CORE DEEP VERIFIED·CORE FRACTURE 판정을 연결하는 최종 코어 연출 구현
- 런별 도전의 순환·진행·단일 보너스 지급, 플레이 상태줄, 완료 컷인과 캐릭터 대사, 결과 판정을 연결하는 MORI REQUEST 구현
- VERIFIED GATE 에 현재 라운드·성공·실패·보상까지 남은 정답 수를 표시하고 더 이상 성공할 수 없는 런을 즉시 구분

- 제한 시간 5초 이하에서 경고색, Web Audio 신호, MORI의 TIME FRACTURE 상태를 동시에 전환
- Cookie Store 비동기 저장과 document.cookie 폴백 구현
- Pretext prepareWithSegments() 와 layoutNextLine() 으로 오염 코어를 피해 흐르는 증거 로그 구현
- Pretext rich-inline API를 이용한 선택지 칩 배치 구현
- 외부 파일 없는 Web Audio 효과음 구현
- GitHub Pages Actions 워크플로 작성

검증

- 여섯 퍼즐 규칙이 한 번씩 등장하고, 시드마다 순서가 달라지면서도 고난도 규칙이 마지막 두 라운드에 유지되며, 라운드마다 정답이 정확히 하나인지 테스트
- 120개 재플레이 시드에서 14개 신호가 모두 출제될 수 있는지, 80개 시드에서 선택지 중복과 체크섬 불일치 수가 정확한지 테스트
- 동일 상태와 시드에서 같은 문제가 생성되는지 테스트
- 점수·난이도·결말 경계값 테스트
- SYNC RECOVERY 가 3연속 이전에는 발동하지 않고 가장 최근 오류를 런당 한 번만 복구하며, 복구 런이 S랭크를 받지 않는지 테스트
- DEEP VERIFY 의 기본 보너스 350점·마지막 코어 보너스 700점과 실패 시 무결성 2칸 손실 경계값 테스트
- MORI REQUEST 3종 순환과 완료 경계, +600점 단일 지급, REQUEST SEALED 상태와 결과 판정 테스트
- 기존 버전 쿠키가 기억 조각 0개인 버전 2로 안전하게 이전되는지 테스트
- 쿠키 데이터 직렬화 시 허용 필드만 보존되는지 테스트
- 프로덕션 빌드, 390×844 모바일 가로 오버플로 0px·위험 선택 타깃 48px, 키보드 전용 6/6 S랭크 및 실제 포인터 클릭만 사용한 마우스 전용 6/6 S랭크 완주와 양 경로의 FINAL CORE +700점 판정 점검

4. 핵심 기술 구조

Pretext 기반 게임 보드

@chenglou/pretext 의 prepareWithSegments() 와 layoutNextLine() 으로 증거 로그를 연속 텍스트 스트림으로 준비합니다. 각 줄의 높이가 중앙 오염 코어와 겹치면 가용 영역을 좌우 슬롯으로 나누고, 앞 줄의 커서부터 다음 슬롯을 이어 배치합니다. 같은 로그도 390px 모바일에서는 폭에 맞춰 더 많은 줄 조각으로 다시 흐릅니다.

선택지에는 @chenglou/pretext/rich-inline 의 prepareRichInline() 과 walkRichInlineLineRanges() 를 사용합니다. 각 선택지는 break: "never" 인 원자적 칩이며, 계산된 위치에 버튼을 절대 배치합니다.

이 방식은 문장 높이를 알아내기 위해 getBoundingClientRect() 로 각 텍스트를 반복 측정하지 않습니다. 컨테이너 폭이 바뀌면 ResizeObserver 가 제공한 폭을 이용해 순수 계산으로 다시 배치합니다.

쿠키 상태 모델

단일 쿠키 state_less_memory_v1 에 다음 화이트리스트 필드만 저장합니다.

```
{
  "version": 2,
  "visits": 1,
  "runs": 0,
  "bestScore": 0,
  "lastEnding": null,
  "policy": "session",
  "fragments": 0
}
```

값은 읽은 뒤 형식·범위·허용 열거형을 다시 검증합니다. 최신 보안 컨텍스트에서는 이벤트 루프를 막지 않는 Cookie Store API를 우선 사용하고, 미지원 브라우저에서는 SameSite=Strict 인 document.cookie 로 전환합니다.

결정적 로컬 추론

방문·테마·시간·입력·탭·화면 폭·네트워크·이전 결말에 완료·런·보존 정책·최고 점수·기억 조각을 더한 14개 사실마다 간접 증거, 참 문장, 거짓 문장, 제3의 오염 문장, 실제 값과 대체 값을 정의합니다. 세션 시드로 사실과 선택지를 섞고 단일 신호 해석에서 시작해 참/거짓 판별, 필드 값 복원, 두 원장 교차검증, 세 필드 체크섬으로 진행합니다. 같은 상태와 시드는 같은 문제를 만들므로 테스트와 재현이 가능합니다.

캐릭터 이미지 프롬프트

캐릭터/MORI_기본이미지_생성_프롬프트.md 에는 초기 성인 여성 설정, 양면적 성격, 색상, 전신 구도와 파일 규격을 기록했습니다. 참여자가 선택한 1024×1536 RGBA PNG는 122KB WebP로 최적화해 UI 배치용 임시 기본본으로 사용합니다.

추가 피드백 이후 캐릭터보다 UI 설정을 먼저 확정했습니다. docs/UI_UX_아트디렉션.md 에서 브라우저 로컬 기억 보관소의 경로·색인·보존·정정 언어를 정하고, 캐릭터/MORI_상황별_캐릭터_설계.md 에서 그 UI에 맞는 기억 사서 실루엣과 복장, 전신·흉상 규격, 17개 런타임 상태, 기존 원본을 잠금 마스킹 편집 프롬프트를 작성했습니다. AI 생성 인상을 줄이기 위해 한 번에 여러 장을 생성하지 않고, 한 기준 흉상에서 표정·손·소품만 편집한 뒤 선화와 손을 수동 정리하도록 제작 절차를 제한했습니다.

5. 외부 에셋과 오픈소스

항목	용도	버전	라이선스·출처
@chenglou/pretext	텍스트 측정 및 rich-inline 배치	0.0.8	MIT, https://github.com/chenglou/pretext
Vite	개발 서버와 프로덕션 번들	8.2.0	MIT, https://github.com/vitejs/vite
OpenAI Codex	개발 보조	사용 환경 제공 버전	https://openai.com/codex/

MORI 기본 이미지는 참여자가 기록된 프롬프트를 사용해 ChatGPT Image에서 생성·선택했습니다. 그 외 외부 이미지, 아이콘, 음원, 영상, 폰트 파일은 사용하지 않았습니다. UI 그래픽은 HTML/CSS로, 효과음은 Web Audio API로 실시간 생성합니다.

6. 검증 명령

```
npm test
npm run build
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=keyboard npm run record
STATELESS_GAME_URL=http://127.0.0.1:5173/ STATELESS_INPUT_MODE=mouse npm run record
```

npm test 는 22개 게임 규칙·쿠키 직렬화 테스트를 실행합니다. npm run build 는 GitHub Pages에 배포할 dist/ 정적 파일을 생성합니다. 기본 녹화 명령은 같은 자동 풀이를 키보드 이벤트와 실제 포인터 클릭으로 각각 수행하고 S랭크·6/6 결과, 가로 오버플로, MORI REQUEST +600점·REQUEST SEALED, DEEP VERIFY 클릭 영역과 FINAL CORE +700점 판정을 검증합니다. STATELESS_PLAY_STYLE=recovery STATELESS_CAPTURE=false 를 추가하면 첫 라운드를 일부러 틀린 뒤 A랭크·6/6·복구 1회를 검증하며, 영상 파일을 만들지 않아 반복 테스트에 사용할 수 있습니다.